

Zustand State Management

Interview Preparation Notes

1 The React State Landscape (The “Why”)

Understanding the problem Zustand solves is critical for architectural interview questions.

1.1 Prop Drilling & Context API

- **Prop Drilling:** Passing state down multiple component layers makes code un-maintainable.
- **Context API:** Solves prop drilling but introduces “Provider Hell” (wrapping the app in multiple `<Provider>` tags) and causes performance issues (entire trees re-render when one Context value changes).

1.2 Redux vs. Zustand

- **Redux:** Highly performant but requires heavy boilerplate (reducers, dispatchers, action types).
- **Zustand:** Minimal boilerplate, a simple `set()` function for updates, no `<Provider>` wrappers, and a tiny bundle size (about 1 KB).

2 Core Zustand Mechanics (The “How”)

Zustand uses a single `create` function to build a custom hook containing both state and actions.

```
import { create } from 'zustand';

const useStore = create((set, get) => ({
  // 1. State
  bears: 0,

  // 2. Actions (Updating state)
  increasePopulation: () => set((state) => ({ bears: state.bears + 1 })),

  // 3. Using get() to read state inside an action
  removeAllBears: () => {
    const currentBears = get().bears;
    if (currentBears > 0) {
```

```

        set({ bears: 0 });
    }
}
}));

```

Listing 1: Basic Zustand Store Setup

2.1 Using Selectors in Components

Selectors extract only specific state, preventing unnecessary re-renders.

```

function BearCounter() {
  // Good: Component only re-renders if 'bears' changes
  const bears = useStore((state) => state.bears);
  return <h1>{bears} around here ...</h1>;
}

function Controls() {
  // Good: Component only re-renders if 'increasePopulation'
  // changes (rarely happens)
  const increasePopulation = useStore((state) => state.
    increasePopulation);
  return <button onClick={increasePopulation}>One Up</button>;
}

```

Listing 2: Component Usage with Selectors

3 Advanced Features & Middleware

Zustand's functionality can be extended without bloating the core library by wrapping the store in middleware.

3.1 Asynchronous Actions

No thunks or sagas are needed. Actions can be standard `async/await` functions.

```

const useUserStore = create((set) => ({
  users: [],
  loading: false,
  fetchUsers: async () => {
    set({ loading: true });
    const response = await fetch('https://api.example.com/users');
    const data = await response.json();
    set({ users: data, loading: false });
  }
}));

```

Listing 3: Async Actions in Zustand

3.2 Persist Middleware

Automatically syncs state to browser storage (`localStorage` by default, or `sessionStorage`).

```
import { create } from 'zustand';
import { persist } from 'zustand/middleware';

const useCartStore = create(
  persist(
    (set) => ({
      cart: [],
      addToCart: (item) => set((state) => ({ cart: [...state.cart, item] })),
    }),
    {
      name: 'cart-storage', // unique name in localStorage
      partialize: (state) => ({ cart: state.cart }), // Only save the cart
    }
  )
);
```

Listing 4: Persisting State

4 Performance & Best Practices

4.1 Preventing Re-renders with `useShallow`

If a selector returns a new object/array, React's strict equality (`===`) will trigger a re-render on every store update, even if the data is identical. Use `useShallow` to perform a shallow comparison.

```
import { useShallow } from 'zustand/react/shallow';

// Without useShallow: re-renders on EVERY store update (new object each time)
// With useShallow: re-renders ONLY if id or name change
const { id, name } = useStore(
  useShallow((state) => ({ id: state.id, name: state.name })))
);
```

Listing 5: Optimizing Object Selectors

4.2 Separating Actions from State

Group actions inside a single object to prevent components that only trigger actions from re-rendering when data changes.

5 Most Probable Interview Questions

1. Why use Zustand over React Context?

Context requires `<Provider>` wrappers and causes re-renders of all consumers when

a value changes. Zustand avoids Provider Hell and uses selectors to re-render only the components that need the changed data.

2. How does Zustand compare to Redux?

Zustand has minimal boilerplate (no reducers, dispatchers, or action types), updates state through a simple `set()` function, and is only around 1 KB in size.

3. How do you handle async API calls in Zustand?

By making the store action an `async` function, awaiting the fetch call, and updating the state using `set()` when the data returns. No middleware like Redux Thunk is required.

4. How do you persist state across page reloads?

By wrapping the store configuration in Zustand's built-in `persist` middleware, which automatically syncs the data to `localStorage`.

5. Can you use a Zustand store outside of React components?

Yes. You can import the store into standard JavaScript files (e.g., utility functions or Axios interceptors) and use `store.getState()` and `store.setState()` to read and update data without React hooks.